

PYGAME : AINCRAD SIMULATOR

1. Project Overview

AINCRAD SIMULATOR : is an idle clicker game built with Python and Pygame, drawing its aesthetic and universe from the anime Sword Art Online. The player faces monsters on various floors of the floating castle Aincrad, defeating them by clicking to deal damage. As monsters die, the player earns Magic Stones (the in-game currency) which can be spent in a shop to permanently upgrade combat stats. The game features a floor progression system, boss raids with time limits, animated sprites, and a full save/load system.

The core gameplay loop is : **click monster → earn gold → buy upgrades → challenge boss → unlock next floor → repeat.**

2. Project Review

Relevant existing project : Cookie Clicker (Orteil, 2013)

Cookie Clicker is the reference idle clicker game → the player clicks a cookie to earn currency, then spends it on upgrades and buildings that generate passive income. It established the genre's core loop of exponential progression and incremental upgrades.

Key improvements :

Cookie clicker	Aincrad Simulator
Only a single-screen	Have multiple floors with layered backgrounds
No combat system	Damage to the entities with upgrades like : damages, critical hits, multiplier
No sprite animation	Sprite animation system
No progression gating	Boss fight to unlock new floors
Same entities	A huge variety of different monsters

3. Programming Development

3.1 Game Concept

AINCRAD SIMULATOR is a cookie-clicker style MMORPG idle game set in a virtual-world called Aincrad. The player controls Louis (Main character), who stands facing enemies on various floors of the castle.

Core mechanics :

- Click on monsters to deal damage and kill them
- Each kill rewards Magic Stones scaled by monster HP and the player's gold multiplier
- Spend Magic Stones in the shop to upgrade stats permanently
- Each floor has its own monster pool with weighted spawn probabilities
- Challenge floor bosses in timed raid encounters with multiple enemy waves
- Defeating a boss unlocks the next floor
- Progress is automatically saved and loaded between sessions

3.2 Object-Oriented Programming Implementation

Class 1 — Game

The central controller that owns the main game loop and all subsystems.

Class 2 — Player

Represents the player character — handles combat stats and position management.

Class 3 — Monster

Represents a clickable enemy with HP and hit detection via mask, and a visual HP bar.

Class 4 — Shop

Manages the upgrade overlay displays purchasable upgrades and applies stat buffs directly to the Player instance.

Class 5 — Floor

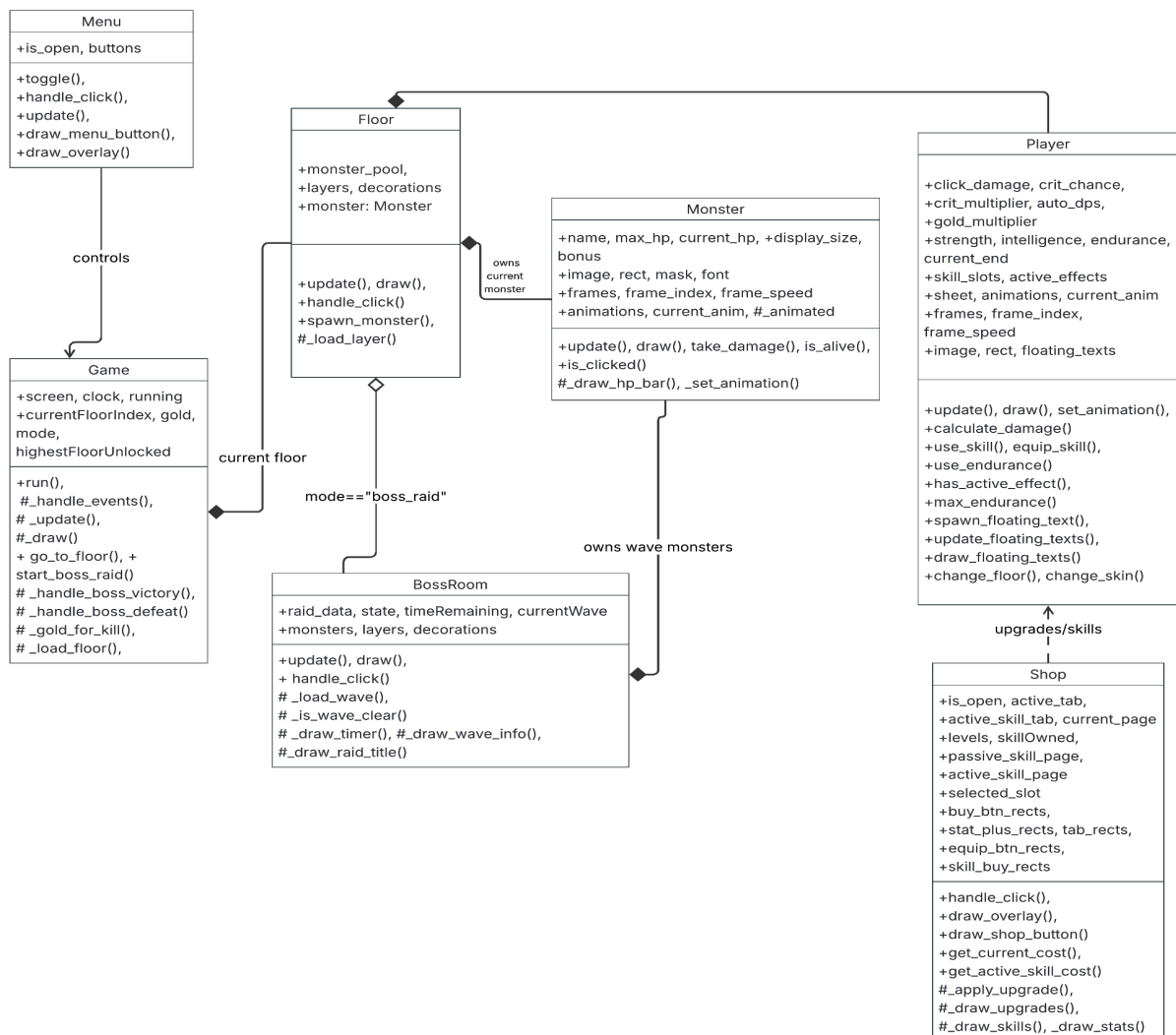
Owns all visual elements of the game floor layer.

Class 6 — BossRoom

Manages a full boss raid encounter multiple monster waves, a countdown timer, and victory/defeat state detection.

Class 7 — Menu

Pause menu overlay with save, load, and logout functionality.



3.3 Algorithms Involved

Mention the algorithms used in the game. If your game incorporates techniques such as pathfinding, sorting, AI behavior, rule-based logic, or event-driven mechanics, describe them here.

1. **Weighted random spawn (`random.choices`)** Monsters don't all have equal spawn probability. Each entry in the floor's monster pool carries a **weight value**. `random.choices(names, weights=weights)` picks a monster proportionally — a Goblin with weight 70 appears roughly 70% of the time against a Slime at 30%.
2. **Critical hit roll** Each click calls `random.random()` which returns a float between 0.0 and 1.0. If this value falls below **`crit_chance`** (e.g. 0.15), the hit is critical and damage is multiplied by **`crit_multiplier`**. This is a simple probability gate algorithm.
3. **Exponential upgrade cost scaling** Each upgrade level costs more than the previous using the formula : **`cost = base_cost × (cost_growth ^ current_level)`** This creates an exponential curve that makes early upgrades cheap and later ones expensive — a standard idle game balancing algorithm.
4. **Boss raid wave management** **`BossRoom`** maintains a **`current_wave`** index and a list of **`Monster`** instances. After each frame, **`_is_wave_clear()`** checks if all monsters in the current wave have **`current_hp == 0`**. If so, the next wave loads. If all waves are cleared, the state becomes "**`victory`**". If **`time_remaining`** reaches 0 first, the state becomes "**`defeat`**".

4. Statistical Data (Prop Stats)

4.1 Data Features

	What is my data going to be used for and why is it good ?	How will you obtain 100 values of this feature data	From which Class and variable i'm going to collect it from	How will you display it ?
Damage per click	It's good to know how click a user did with a certain damage value and it allows identifying outliers in damage output that may indicate unbalanced upgrade combinations	Integer per click event	I will collect the data from the class Player using method : <code>calculate_damage</code> and collect it from the variable <code>int(damage)</code>	Using Line chart to the show the user damage evolution over time
Monster defeated	It's a good data because we can compute mean damage per floor, compare floors, or calculate kill rate per minute	1 row per kill	I will collect it from the Class Monster and the method : <code>is_alive</code> incrementing with variable <code>deaths (int)</code>	Bar Chart to compare which floor we killed the most monsters, useful for the comparison.
Gold spent	Recording the purchase that my user buy can help me to understand which upgrade combination can be unbalanced and how many gold did he have to spend to obtain it	1 row per purchase	I will use the Class Game and to calculate the cost of an upgrade i will do <code>(old_gold - self.gold)</code> when an upgrade is purchased	Bar chart to see which upgrade did the user spend the most money on.
Active-skill activation	can reveal if one skill dominates usage, suggesting the others may need rebalancing	1 row per activation	Class player, return <code>(damage,skill_id)</code> or <code>(none,skill_id)</code>	Pie chart to compare the area of usage of my differents skills
How many time spent on the game	Record total playtime, can be useful to compare with other recorded data (for example for stating that in : <code>xxhours:xxmin:xxsec`</code> , <code>xx clicks where made</code>)	1 row per 10 seconds interval (can be reached in ≈ 17 minutes : $((10 * 100) / 60)$	Class Game, will be recorded by the var <code>self._session</code> that i increment in <code>_update() += dt</code>	Summarization statistics. Stating how many time did the user spend on the game in total

4.2 Data Recording Method

All statistical data will be stored in a **CSV file (saves/stats.csv)** alongside the existing JSON save file (to save my progression). CSV is chosen because :

- It is human-readable and easy to inspect
- Python's built-in **csv** module requires no external dependencies
- It is directly importable into Excel, Google Sheets, or pandas for analysis

saves/

|— savegame.json ← game state (existing)

|— stats.csv ← one column per stat updated separately

4.3 Data Analysis Report

Data will be analyzed using **Python with pandas and matplotlib** in a separate `analysis.py` file. statistical measures and visualizations are planned like this :

	<u>Feature name</u>	<u>Graph objective</u>	<u>Graph type</u>	<u>X-axis</u>	<u>Y-axis</u>
Graph 1	Damage per click	Show damage evolution over time (time series)	Line graph	Time	Damage Value
Graph 2	Monster defeated	Compare kills across floors (proportion)	Bar graph	Floor	Number of monster killed
Graph 3	Gold spent	Compare spending per upgrade (proportion)	Bar graph	Upgrade type	Gold spent
Graph 4	Active-skill activation	Compare usage area across differents skills (proportion)	Pie graph	—	Sections = SKILL ID & NAME

4.3.1 Table of statistical Values

<u>Feature</u>	<u>Statistical Value</u>
Damage dealt	Mean, Median, Standard Deviation, Min, Max
Monster defeated	Mean, Median, Standard Deviation, Min, Max
Gold spent	Mean, Median, Standard Deviation, Min, Max
Active-skill activation	Mean, Median, Standard Deviation, Min, Max
Game time	Mean, Median, Standard Deviation, Min, Max

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Expand shop feature, add active skills
3	10	Balance my game (Mob health, damage,upgrade)
4	11	Polish design and fix bugs/error
5	12	Statistical data tracking feature
6	13	Review my project
7	14	Submission week (Draft)

6. Document version

Version: 1.2

Date: 11 March 2025

Editor: Hugo Desvignes

Date	Name	Feedback
19/03	Peraya	Your game concept is strong and I like that. However, there are some issues you would need to resolve. Major change needed: - Section 3.2: You give me the UML diagram which is good but I cannot see the detail of it. You also need to list and describe at least five classes you will implement in the program. For each, you must define its role, key attributes, and methods. - Section 4.1: Your plan for Boss raid attempt and Session time per floor is risky. To get 100 records for Boss attempts, a player would need to fight a boss 100 times. Similarly, for Session time per floor, which I do not understand what you want to collect. Consider tracking more frequent events to ensure you hit the >100 records requirement easily during a standard playtest.
30/03	Peraya	Missing Statistical Values Table: You provided a great UML diagram and class descriptions, but you are still missing the Table of Statistical Values (showing Mean, Median, Std. Deviation, Min, and Max) for your 5 features. Overall, good work!